

# GRIDIRON

An NFL Player Projection Pipeline on Kubernetes

---

Project Deliverable 1 — Architecture

**Dan Gannon**

## Chapter 1: Vision

### 1.1 Overview

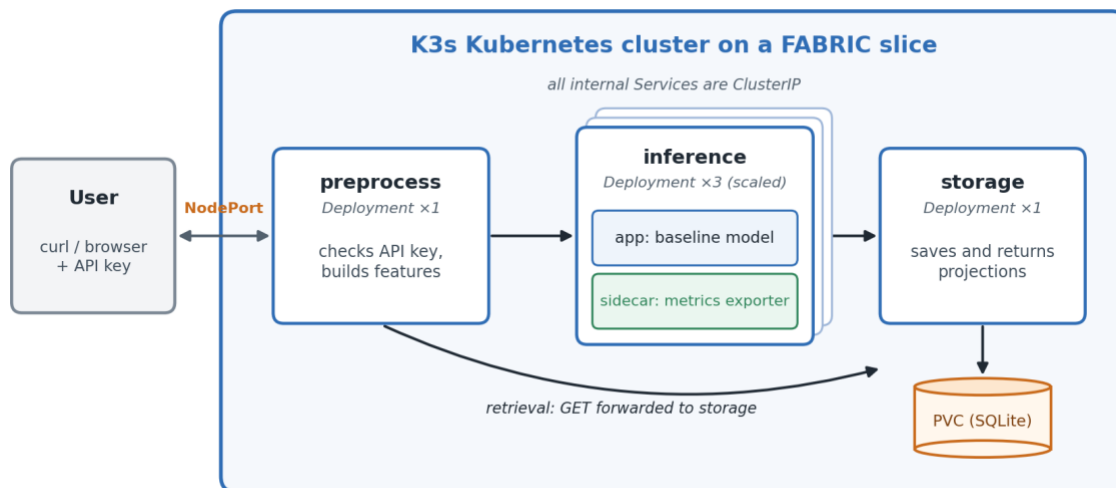
I want to make a project called Gridiron that you send it a player and a week, and it returns projected yards, catches and touchdowns based on that player's recent games.

The goal of this project is to build Gridiron as a small pipeline on Kubernetes. The three stages of preprocessing turns a player's recent statistics into model inputs, inference produces the projection, and storage saves the result so it can be retrieved later. The inference stage is scaled to several different models. The model is going to be basic, to focus on the orchestration of the project.

### 1.2 User Stories

- **Get a projection:** as a user, I send a player ID and week and receive that player's projected stats.
- **Look it up later:** as a user, I retrieve a projection I requested earlier by its ID.
- **See the week:** as a user, I list every projection saved for a given week.

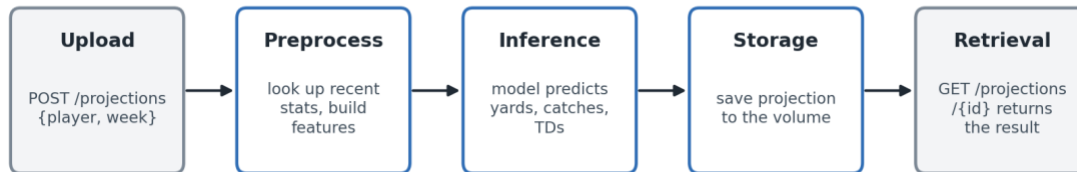
### 1.3 Architecture at 10,000 Feet



Gridiron will run as three Deployments on a small K3s Kubernetes cluster inside a FABRIC slice. The preprocess pod is the only component users can reach, through a NodePort Service. It checks the user's API key, builds the input features, and calls the inference Service. Inference is the part that gets scaled, so it runs three replicas, and each pod has two containers: the model app and a metrics-exporter sidecar that reports request counts and response times. Inference passes its result to the storage pod, which writes it to a PersistentVolumeClaim so that saved projections survive pod restarts. All communication inside the cluster uses ClusterIP Services.

## Chapter 2: Proposed Implementation

### 2.1 Data Flow



A user sends POST `/projections` with a player ID and week. Preprocess looks up that player's recent games in a small, pinned CSV of public nflverse weekly statistics (included in its container image) and builds the features. Inference returns the projected stats, and storage saves them with a new ID. The user can then fetch the result with GET `/projections/{id}`, which preprocess forwards to storage.

### 2.2 Components

| Pod        | Containers            | Role                                                                                                                      |
|------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
| preprocess | app                   | External entry point. Checks the API key, builds features from the stats CSV, calls inference, forwards reads to storage. |
| inference  | app + metrics sidecar | Baseline model: average of the player's last four games. The sidecar exposes request count and latency.                   |
| storage    | app                   | Saves projections to a SQLite file on the persistent volume and returns them on request.                                  |

Table 1. Pods

Each service is a small Python (FastAPI) app packaged as a Docker image with a pinned version tag.

### 2.3 Services and Scaling

| Service    | Type      | Reason                                       |
|------------|-----------|----------------------------------------------|
| preprocess | NodePort  | The single external entry point              |
| inference  | ClusterIP | Internal only; load-balances across replicas |
| storage    | ClusterIP | Internal only                                |

Table 2. Service choices

**Scaling target: inference.** It does the repeated work and keeps no state, so it can be scaled freely. It will run three replicas, and I will show self-healing by deleting one inference pod and watching Kubernetes recreate it. **Authentication:** users send an X-API-Key header, which preprocess checks against a key stored in a Kubernetes Secret. Requests with a missing or wrong key get a 401 response.

### 2.4 API Contracts

| Endpoint                       | Description                                                 |
|--------------------------------|-------------------------------------------------------------|
| POST <code>/projections</code> | Body: player ID and week. Returns the new projection (201). |

| Endpoint               | Description                                              |
|------------------------|----------------------------------------------------------|
| GET /projections/{id}  | Returns one saved projection (200), or 404 if not found. |
| GET /projections?week= | Returns all projections saved for that week.             |

Table 3. External API (served by preprocess)

```
POST /projections      X-API-Key: <key>
{ "player_id": "00-0012345", "week": 4 }

201 Created
{ "id": 17, "player_id": "00-0012345", "week": 4,
  "rec_yds": 78.4, "rush_yds": 2.0, "receptions": 6.1, "td": 0.5 }
```

## 2.5 Persistence Plan

Only the storage pod keeps data. It mounts one 1 Gi PersistentVolumeClaim (using K3s's built-in local-path storage) at /data. Projections are stored in /data/projections.db (SQLite). The stats CSV is read-only and ships inside the preprocess image, so it does not need a volume. Persistence will be verified by restarting the storage pod and confirming that saved projections are still returned.

## 2.6 Risks and Test Plan

- **FABRIC slice expires:** keep every manifest in the repository so the stack can be rebuilt with `kubect1 apply -f ..`
- **Pod-to-pod networking across FABRIC nodes:** test with a simple workload before deploying the app.
- **Slow responses:** keep the model tiny and set a request timeout in preprocess.

| Test          | Input                            | Expected                            |
|---------------|----------------------------------|-------------------------------------|
| Happy path    | Valid player and week, valid key | 201 and projection returned         |
| Invalid input | Unknown player ID or bad week    | 404 / 422 error                     |
| Bad key       | Missing or wrong API key         | 401                                 |
| Many requests | 50 requests in a row             | All succeed; spread across replicas |
| Restart       | Delete storage pod, then GET     | Saved projection still returned     |